UNIVERSIDAD DA VINCI DE GUATEMALA

FACULTAD DE INGENIERÍA

INGENIERÍA EN SISTEMAS

ESTRUCTURA DE DATOS

ING. BRANDON CHITAY

PRIMER PARCIAL

ANÁLISIS DE RENDIMIENTO DE ALGORITMOS

https://youtu.be/oyiJzJ3rZEg

María Elena García Valencia

202405749

03/03/2026

INTRODUCCION

La notación Big-O es una herramienta matemática esencial en la ingeniería de sistemas que nos permite medir qué tan eficiente es un algoritmo al evaluar cómo aumenta su tiempo de ejecución o el uso de memoria cuando crece la cantidad de datos (n). Su función es darnos una métrica predecible que no dependa del hardware, permitiéndonos comparar diferentes soluciones lógicas para anticipar dónde podrían ocurrir retrasos. En este análisis, la utilizaremos para identificar comportamientos lineales, cuadráticos y exponenciales, ayudándonos a elegir el método óptimo para cada tarea.

DESCRIPCIÓN DE LOS ALGORITMOS

1. Factorial (A1)

Este algoritmo calcula el producto de todos los números enteros positivos desde 1 hasta n. La versión iterativa utiliza un ciclo definido para acumular el resultado, mientras que la versión recursiva se va llamando a sí misma con subtareas más pequeñas hasta llegar al caso base. Al final, los dos tienen una complejidad lineal O(n), porque mientras más grande es el número de datos de entrada, el tiempo aumenta de forma proporcional.

2. Fibonacci (A2)

Este algoritmo genera una secuencia donde cada número es la suma de los dos anteriores. La versión iterativa es altamente eficiente al mantener un registro de los valores previos en un solo ciclo. Por el contrario, la versión recursiva presenta un crecimiento exponencial O(2^n), debido a que recalcula los mismos valores repetidamente, lo que dispara los tiempos de ejecución drásticamente al aumentar n.

3. Búsqueda Lineal (A3)

Es un método de búsqueda que recorre secuencialmente cada elemento de un arreglo hasta encontrar el valor objetivo o finalizar la lista. Su rendimiento se evalúa en el "peor caso", que ocurre cuando el elemento no está presente y el algoritmo debe realizar n comparaciones. Tanto la implementación iterativa como la recursiva mantienen una complejidad lineal O(n), siendo dependientes directos del tamaño del conjunto de datos.

4. Ordenamiento Burbuja (A4)

Este algoritmo ordena la lista comparando números en parejas y cambiándolos de lugar si están en el orden incorrecto, repitiendo esto hasta que todo quede organizado. Como usa ciclos dentro de otros ciclos (anidados), su complejidad es cuadrática O(n^2). Esto significa que el tiempo aumenta de forma proporcional al cuadrado de los datos, haciendo que no sea muy eficiente cuando manejamos gran cantidad de datos.

FRAGMENTOS DE CÓDIGO EXPLICADO:

```java
public class Medidor {
    // Registra el tiempo en nanosegundos y retorna milisegundos
    public static double calcularPromedio(Runnable algoritmo) {
        long sumaNanos = 0;
        int repeticiones = 5;

        for (int i = 0; i < repeticiones; i++) {
            long inicio = System.nanoTime();
            algoritmo.run();
            long fin = System.nanoTime();
            sumaNanos += (fin - inicio);
        }

        double promedioNanos = (double) sumaNanos / repeticiones;
        return promedioNanos / 1_000_000.0;
    }
```

Este método automatiza las 5 repeticiones requeridas para cada prueba. Captura el tiempo en nanosegundos usando System.nanoTime() y luego lo convierte a milisegundos para que los resultados sean fáciles de leer y graficar.

```java
// A2 - Recursivo: O(2^n)
public static long calcularRecursivo(int n) {
    if (n <= 1) return n;
    return calcularRecursivo(n - 1) + calcularRecursivo(n - 2);
}
```

Aquí vemos la recursión doble. Por cada llamada a la función, se generan otras dos llamadas más, lo que crea un efecto de 'bola de nieve' o crecimiento exponencial $O(2^n)$ que se nota drásticamente en los tiempos de ejecución. Esto explica por qué la gráfica de Fibonacci se dispara hacia arriba.

```java
// A4 - Recursivo: O(n^2)
public static void ordenarRecursivo(int[] arr, int n) {
    if (n == 1) return; // Caso base

    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }
    // Llamada para el resto del arreglo
    ordenarRecursivo(arr, n - 1);
}
```

Al tener un ciclo for dentro de otro, el algoritmo tiene que recorrer el arreglo muchísimas veces. Esta estructura es la que define su complejidad O(n^2), haciendo que el tiempo suba rápido al aumentar el tamaño del arreglo.

TABLA DE TIEMPOS MEDIDOS

FACTORIAL

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0,018968	0,000224
10	0,000096	0,000124
15	0,000144	0,000256
20	0,000116	0,000232
25	0,000112	0,000000
30	0,000164	0,000000

FIBONACCI

HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo: O(2^n)

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0,023440	0,000332
10	0,000088	0,005012
15	0,000276	0,002520
20	0,000156	0,009620
25	0,000184	0,068700
30	0,000196	0,729492

BÚSQUEDA LINEAL

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo: O(n)

n	Iterativo (O(n))	Recursivo (O(n))
100	0,024240	0,001452
500	0,001324	0,001680
1000	0,002544	0,003352
5000	0,012328	0,013584
10000	0,028684	0,013100

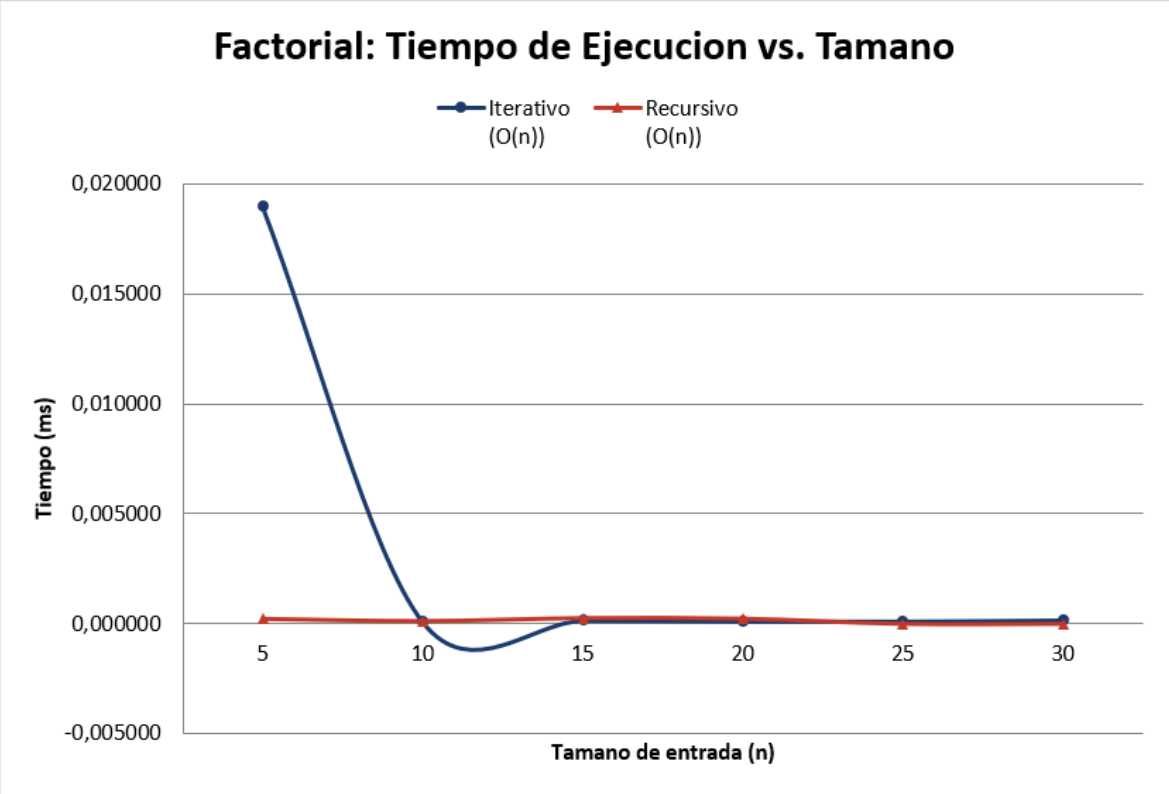
BURBUJA

HOJA — A4 - Burbuja | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n^2)	Big-O Recursivo:

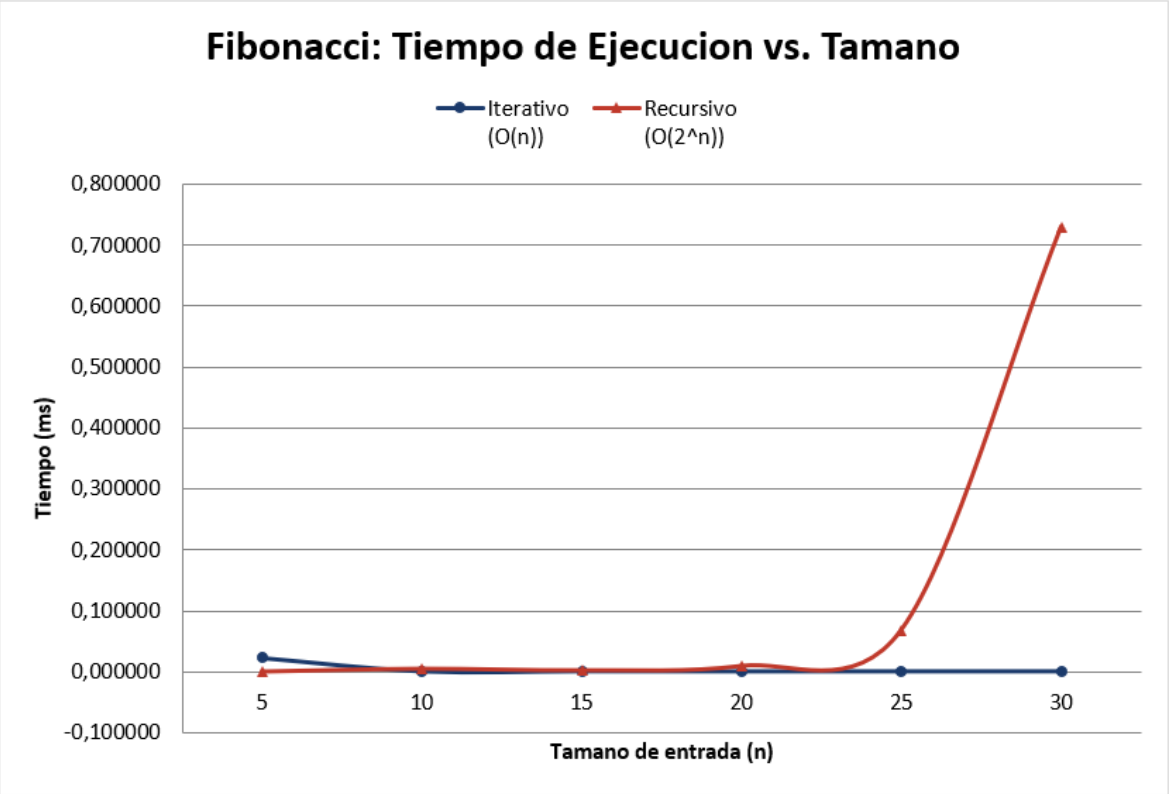
n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0,058008	0,017228
500	0,173584	0,108532
1000	0,292540	0,360264
5000	4,957500	4,465892
10000	24,249644	21,367620

GRÁFICAS

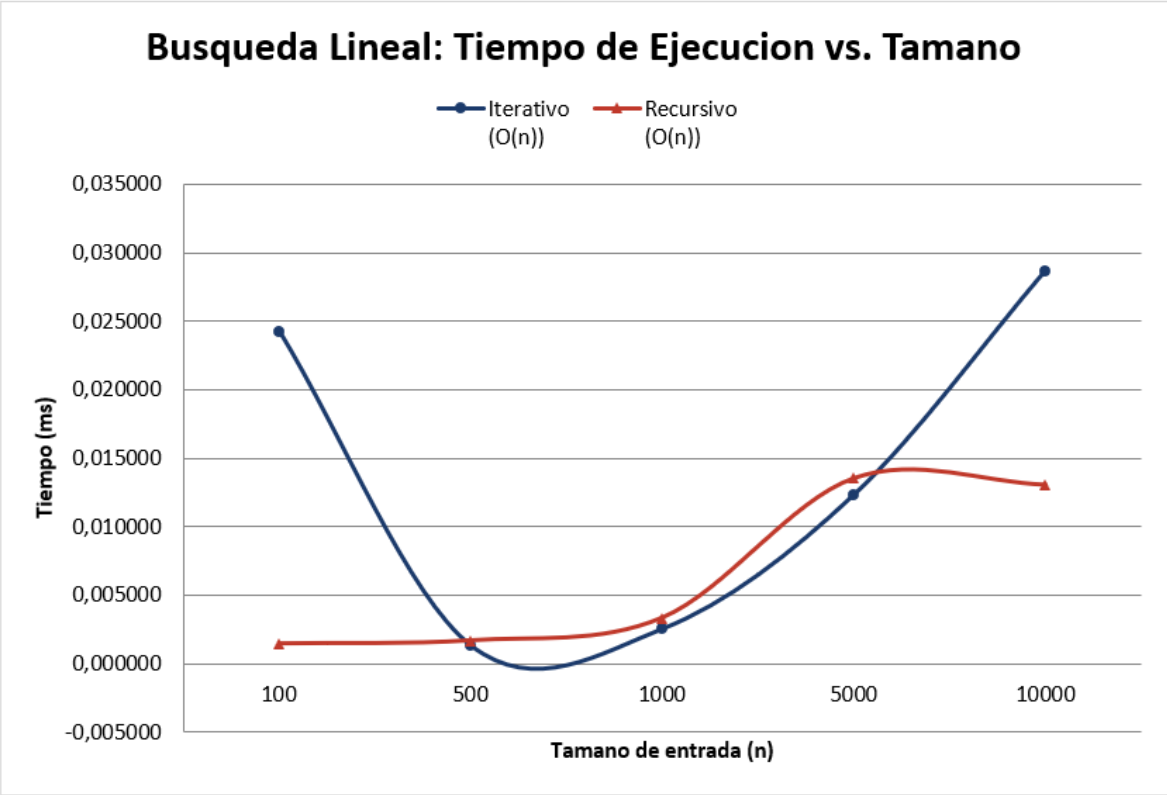
FACTORIAL



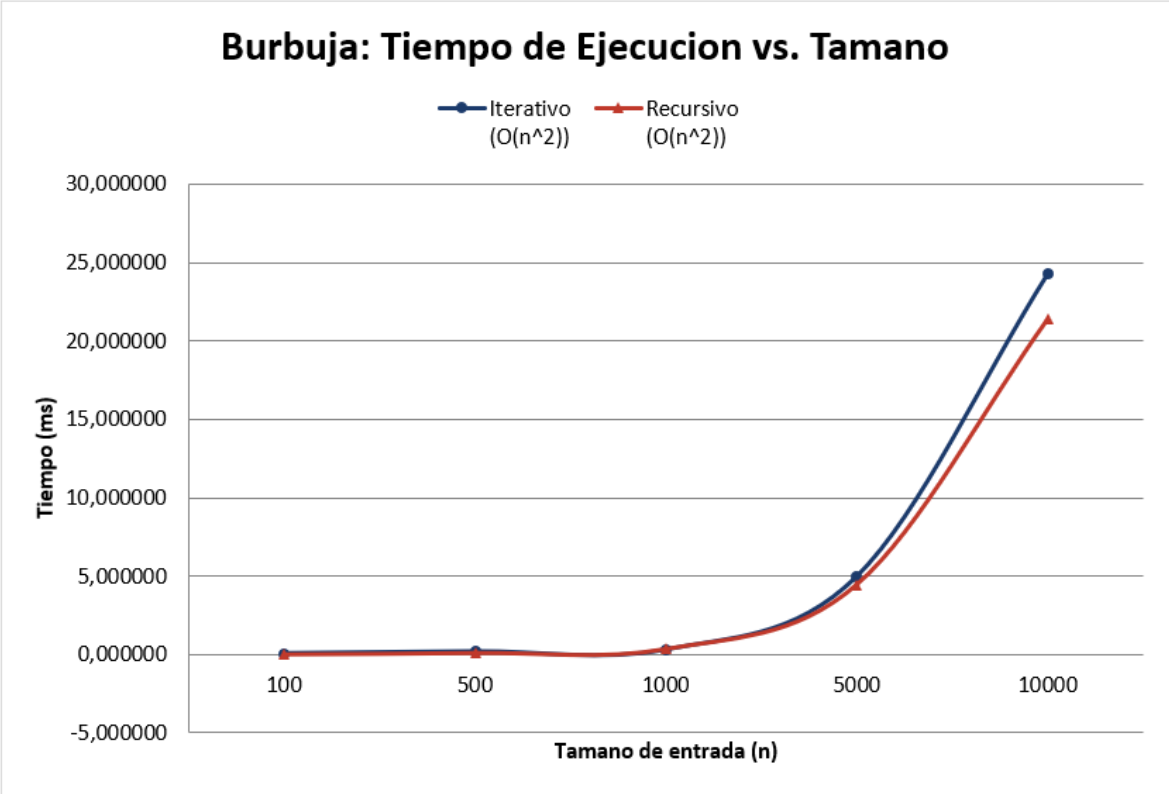
FIBONACCI



BÚSQUEDA LINEAL



BURBUJA



ANÁLISIS DE RESULTADOS

El análisis demuestra que, aunque la recursividad ofrece un código más elegante, su impacto en el rendimiento varía según el algoritmo. Mientras que en modelos lineales como Factorial o Búsqueda el tiempo es similar al iterativo, en Fibonacci la recursión simple dispara el costo computacional de forma exponencial. Esta diferencia confirma que la elección del método debe basarse en la eficiencia del manejo de memoria y procesador, no solo en la estética del código.

CONCLUSIONES

- Los métodos iterativos son más eficientes en rendimiento y memoria al evitar la acumulación de llamadas en el stack.
- Se comprobó que la recursión en Fibonacci escala de forma exponencial O(2^n), siendo inviable para valores grandes.
- El algoritmo de Burbuja confirmó su complejidad cuadrática O(n^2), pues el tiempo se dispara al aumentar el tamaño del arreglo.
- La recursión es útil por su legibilidad en problemas lineales O(n), pero debe evitarse en procesos de alto costo computacional.

REFERENCIAS BIBLIOGRÁFICAS

- Juárez, G., & Martínez, I. (2013). Estructura de Datos. Instituto Politécnico Nacional (ESCOM). Recuperado de https://www.escom.ipn.mx/docs/oferta/matDidacticoISC2009/EDts/Libro_EstructuraDatos.pdf

- Groner, L. (2016). Learning JavaScript Data Structures and Algorithms (2da ed.). Packt Publishing. Recuperado de https://github.com/GauravWalia19/Free-Algorithms-Books/blob/main/Library/src/JAVASCRIPT/Learning-JavaScript-Data-Structures-and-Algorithms-Second-Edition.pdf